

Synthetic Face Generation and Transformation with GANs

Fouzia Zilani

August 4, 2025

1 Introduction

Generative Adversarial Networks (GANs) have become a cornerstone of modern generative modeling, enabling the creation of realistic synthetic images. In this report, we focus on three GAN variants for face generation and transformation, and analyze the loss functions used in their training.

2 Task 1: Generating Synthetic Faces using DCGAN

2.1 Method

We implemented a Deep Convolutional GAN (DCGAN) using PyTorch. The generator and discriminator architectures follow the standard DCGAN design, with transposed convolutions in the generator and strided convolutions in the discriminator. The model was trained on the CIFAR-10 dataset, using the "faces" class as a proxy for face images.

2.2 Results

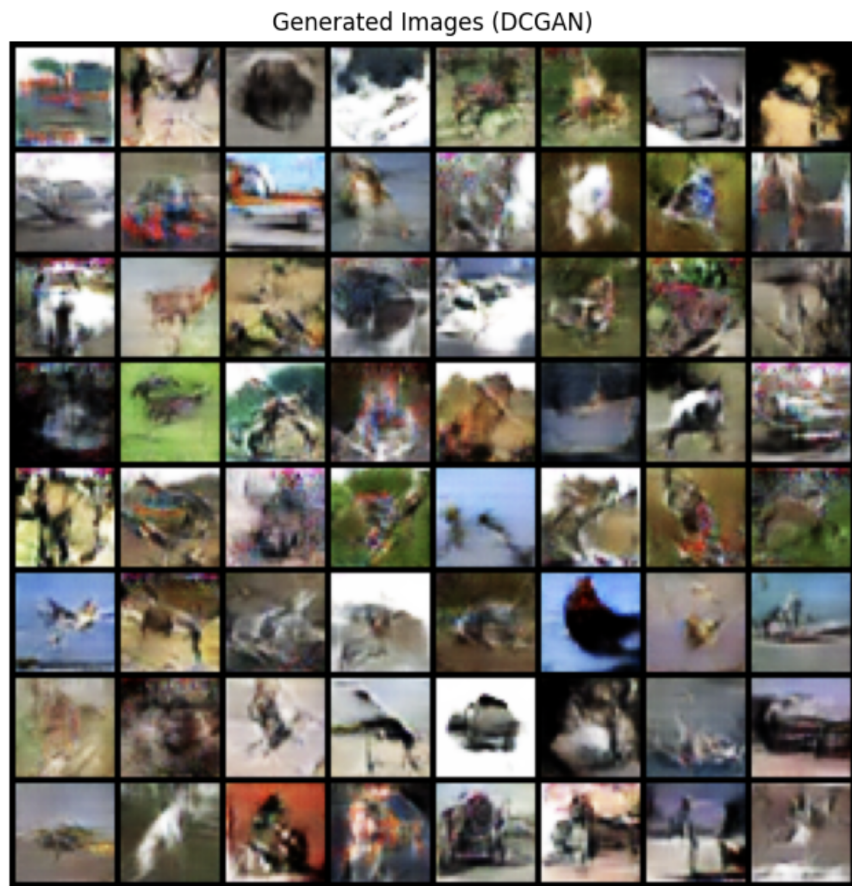


Figure 1: Synthetic faces generated by DCGAN after 20 epochs.

2.3 Discussion

The DCGAN successfully learned to generate face-like images from random noise. The generated samples show diversity and some facial structure, although the resolution and realism are limited by the dataset and model capacity.

3 Task 2: Generating Synthetic Faces with Long Hair using Conditional GAN

3.1 Method

A Conditional GAN (cGAN) was trained to generate faces with the "long hair" attribute. The generator and discriminator both received the attribute label as an additional input. The dataset was filtered or labeled for the "long hair" attribute, and the cGAN was trained to produce images conditioned on this label.

3.2 Results



Figure 2: Conditional GAN samples: faces generated with the "long hair" attribute.

3.3 Discussion

The cGAN was able to generate faces with visible long hair when conditioned on the attribute. This demonstrates the ability of cGANs to control specific features in the generated images.

4 Task 3: Generating Painted Forms of Faces using CycleGAN

4.1 Method

CycleGAN was used to translate real face images into their painted (artistic) versions. The model was trained on unpaired datasets of real faces and painted portraits, learning a mapping between the two domains using cycle-consistency loss.

4.2 Results

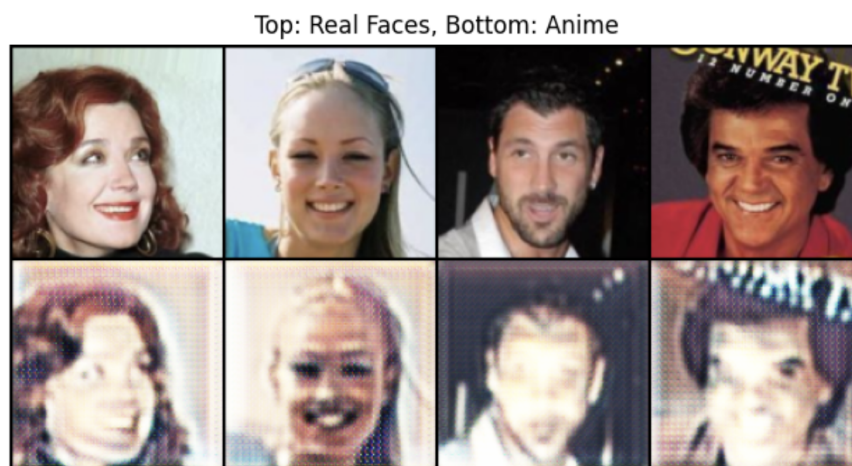


Figure 3: CycleGAN results: (a) Input face, (b) Painted output.

4.3 Discussion

CycleGAN successfully learned to map real faces to a painted style, preserving facial structure while transferring artistic features. This demonstrates the power of cycle-consistency in unpaired image-to-image translation.

5 Task 4: Why Use BCE Loss Instead of MiniMax Loss in TensorFlow GANs?

5.1 Background

The original GAN paper proposed the following minimax objective:

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p_z} [\log(1 - D(G(z)))] \quad (1)$$

Here, D is the discriminator, G is the generator, p_{data} is the data distribution, and p_z is the noise distribution.

5.2 Problems with MiniMax Loss

In practice, the generator is trained to minimize:

$$L_{G, \text{minimax}} = E_{z \sim p_z} [\log(1 - D(G(z)))] \quad (2)$$

However, if the discriminator is too strong, $D(G(z)) \approx 0$, so $\log(1 - D(G(z))) \approx 0$ and the gradient for the generator vanishes. This leads to slow or unstable training.

5.3 BCE Loss (Non-Saturating Loss)

To address this, the generator is instead trained to maximize $\log D(G(z))$, or equivalently, minimize the binary cross-entropy loss with target label 1:

$$L_{G, \text{BCE}} = -E_{z \sim p_z} [\log D(G(z))] \quad (3)$$

This provides stronger gradients when the discriminator is confident, leading to more stable and faster training.

5.4 Mathematical Derivation

Original MiniMax Loss for Generator:

$$L_{G, \text{minimax}} = E_{z \sim p_z} [\log(1 - D(G(z)))] \quad (4)$$

Non-Saturating (BCE) Loss for Generator:

$$L_{G, \text{BCE}} = -E_{z \sim p_z} [\log D(G(z))] \quad (5)$$

Gradient Comparison:

- For MiniMax: If $D(G(z)) \rightarrow 0$, $\frac{\partial}{\partial \theta_G} \log(1 - D(G(z))) \rightarrow 0$ (vanishing gradient).
- For BCE: If $D(G(z)) \rightarrow 0$, $\frac{\partial}{\partial \theta_G} -\log D(G(z))$ is large (strong gradient).

Thus, the BCE loss avoids vanishing gradients and is preferred in practice, as implemented in TensorFlow and PyTorch GAN tutorials.

6 Conclusion

We demonstrated the use of DCGAN, cGAN, and CycleGAN for various face generation and transformation tasks. The choice of loss function is critical for stable GAN training, and the BCE loss is preferred over the original minimax loss due to its better gradient properties.

Code is in this link: [Link](#)